

Cafe Rewards Dashboard

Data Cleaning & DAX Measures Documentation

Project Overview

This document describes the data cleaning, transformation, data integration, and DAX measures developed for the Cafe Rewards Dashboard.

The raw dataset contained customer information, offer details, and event-level activity such as offers received, offers viewed, offers completed, and transactions. Power Query was used to clean and prepare the data, while DAX was used to create business metrics and KPIs for the dashboard.

1. Data Cleaning & Transformation

1.1 Events Table Transformation

The Events table contained different types of customer activities, including:

- Offer Received
- Offer Viewed
- Offer Completed
- Transaction

The value field contained different information depending on the event type. Therefore, the Events table required significant transformation before it could be used for analysis.

1.2 Separating Transaction Amount and Offer ID

Conditional columns were created to separate transaction-related information from offer-related information.

For transaction events, the transaction amount was extracted from the value_extracted field.

Logic:

```
If event = "transaction"  
Then value_extracted  
Else null
```

This created the Transaction_amount column.

This allowed transaction spending to be analyzed separately from offer activity.

1.3 Extracting Offer ID

The raw offer information was stored inside the value_extracted field in a text format.

The Text Between Delimiters transformation was used to extract the actual offer ID.

The required value was extracted between the relevant offer ID delimiters.

This created the clean_offer_id column.

This step was important because the raw Events table did not contain a directly usable offer ID column.

1.4 Creating Reference Tables

Reference tables were created from the Events table to support the identification of relationships between customer offer activity and transactions.

The relevant event records were filtered, particularly:

- event = "offer completed"

The reference data retained important fields such as:

- Customer ID
- Event
- Time
- Clean Offer ID

These reference tables were used to identify the offer associated with subsequent transactions.

1.5 Identifying Transaction-Associated Offer IDs

The Events data was merged with the relevant reference table to identify the offer associated with a transaction.

The matching information was based on the customer and offer-completion relationship.

A conditional column was then created:

```
If event = "transaction"
Then matched_clean_offer_id
Else null
```

This created the transaction_offer_id column.

The purpose of this transformation was to connect transactions with the offers that influenced those transactions.

1.6 Creating Final Offer ID

A final offer ID column was created to provide one consistent offer identifier across the customer journey.

Logic:

```
If event = "transaction"
Then transaction_offer_id
Else clean_offer_id
```

This created the final_offer_id column.

The final offer ID could therefore be used across different stages of the customer journey, including:

- Offer Received
- Offer Viewed
- Offer Completed
- Transaction

This was one of the most important transformations in the project because it allowed offer activity and transaction activity to be connected.

2. Integrating Offer Information

After creating the final_offer_id, the Events table was merged with the Offers table.

The tables were matched using:

```
events_final[final_offer_id]
      ↓
offers[offer_id]
```

A Left Outer Join was used so that the event records were retained while matching offer information was brought into the dataset.

The following offer attributes were added:

- Offer Type
- Difficulty
- Reward
- Duration
- Channels

This allowed the dashboard to analyze offer performance by offer characteristics.

3. Customer Data Cleaning

3.1 Age Validation

The Customers table contained some unrealistic age values above 100.

These records were excluded from the analysis because they also contained incomplete demographic information, including missing income and gender information.

The analysis was therefore restricted to customers with:

- Age < 100

This improved the reliability of demographic analysis.

3.2 Age Column and Age Groups

An Age column was prepared and an Age Group classification was created.

Age groups were used in the dashboard to analyze:

- Customer distribution
- Offer-associated spending
- Customer demographics

The Age Group field allows management to understand which customer segments represent the largest portion of the customer base.

3.3 Income Band

An Income Band column was created to categorize customers according to income.

Income Range	Income Band
Less than 40,000	Lower Earner
40,000 to 80,000	Middle Earner
Greater than 80,000	Higher Earner

This classification was created to support customer segmentation and demographic analysis.

4. Customer and Event Data Integration

After cleaning the Customers table and removing customers with age values above 100, the cleaned customer data was merged with the events_final table.

The tables were matched using:

- customer_id

An Inner Join was used.

The purpose of the Inner Join was to retain customers that had corresponding event activity in the Events dataset.

The resulting analytical dataset contained the cleaned customer information together with important event and offer information such as:

- Customer ID
- Age
- Age Group
- Gender
- Income
- Income Band
- Event
- Time
- Transaction Amount
- Clean Offer ID
- Final Offer ID
- Offer Type
- Difficulty
- Reward
- Duration
- Channels

This prepared the data for Power BI reporting and dashboard development.

5. DAX Measures

DAX measures were created in Power BI to convert the cleaned data into meaningful customer, offer, transaction, and spending metrics.

The following measures were used in the dashboard.

5.1 Customer & Transaction Measures

1. Consumers

DAX:

```
Consumers = COUNT(customers[customer_id])
```

Purpose:

Counts the total number of customers in the cleaned Customers table.

2. Total Offer ID

DAX:

```
Total Offer ID = COUNTROWS(events_final)
```

Purpose:

Counts the total number of records in the final Events table.

3. Total Transactions

DAX:

```
Total Transactions =  
CALCULATE(  
    COUNTROWS(events_final),  
    events_final[event] = "transaction"  
)
```

Purpose:

Counts all transaction events in the final Events table.

4. Offer Attribute Transaction

DAX:

```
Offer Attribute Transaction =  
CALCULATE(  
    COUNT(events_final[final_offer_id]),  
    events_final[event] = "Transaction"  
)
```

Purpose:

Counts transactions that have an associated offer ID, allowing offer-associated transactions to be separated from non-offer transactions.

5. Non Offer Transaction

DAX:

```
Non offer transaction =  
[Total Transactions] - [Offer Attribute Transaction]
```

Purpose:

Calculates transactions that were not associated with an offer.

6. Amount from Transactions

DAX:

```
Amount from Transactions =  
SUM(events[Transaction_amount])
```

Purpose:

Calculates the total monetary value of transactions.

5.2 Offer Journey Measures

7. Offer Received

DAX:

```
Offer Received =  
CALCULATE(  
    COUNT(events_final[final_offer_id]),  
    events_final[event] = "offer received"  
)
```

Purpose:

Counts the number of offers received by customers.

8. Offer Viewed

DAX:

```
Offer Viewed =  
CALCULATE(  
    COUNT(events_final[final_offer_id]),  
    events_final[event] = "offer viewed"  
)
```

Purpose:

Counts the number of offers viewed by customers.

9. Offer Completed

DAX:

```
Offer Completed =  
CALCULATE(  
    COUNT(events_final[final_offer_id]),  
    events_final[event] = "offer completed"
```

```
)
```

Purpose:

Counts the number of offers completed by customers.

10. View Rate

DAX:

```
View Rate =  
DIVIDE(  
    [Offer Viewed],  
    [Offer Received]  
)
```

Purpose:

Measures the percentage of received offers that were viewed.

11. Completion Rate

DAX:

```
Completion Rate =  
DIVIDE(  
    [Offer Completed],  
    [Offer Received]  
)
```

Purpose:

Measures the percentage of received offers that were completed.

12. Transaction Rate Through Offer

DAX:

```
Transaction rate through offer =  
DIVIDE(  
    [Offer Attribute Transaction],  
    [Offer Received]  
)
```

Purpose:

Measures the proportion of received offers that resulted in an associated transaction.

5.3 Transaction Amount Measures

13. Amount from Transaction Offer

DAX:

```
Amount from transaction offer =  
CALCULATE(  
    SUM(events_final[Transaction_amount]),  
    events_final[event] = "transaction",  
    NOT ISBLANK(events_final[final_offer_id])  
)
```

Purpose:

Calculates the transaction amount generated through offer-associated transactions.

14. Amount from Transaction

DAX:

```
Amount from transaction =  
[Amount from Transactions] - [Amount from transaction offer]
```

Purpose:

Calculates the transaction amount that was not associated with an offer.

15. Offer Associated Transactions

DAX:

```
Offer associated Transactions =  
DIVIDE(  
    [Offer Attribute Transaction],  
    [Total Transactions]  
)
```

Purpose:

Calculates the percentage of total transactions that were associated with an offer.

5.4 DAX Functions Used

The main DAX functions used in this project were:

- COUNT – counts values in a column.
- COUNTROWS – counts rows in a table.
- CALCULATE – applies specific filter conditions while calculating a measure.
- SUM – calculates total transaction amounts.

- DIVIDE – performs division safely and is used for calculating rates and percentages.
- ISBLANK – identifies blank values when determining offer-associated transactions.

These measures were used to create the dashboard metrics, offer funnel analysis, transaction analysis, and spending comparisons.